# Edge Deployment Report

BRISC 2025 Brain Tumor MRI – Inference and Edge Deployment

August 27, 2026

How the BRISC brain tumor classification and segmentation models were taken from trained checkpoints to working edge deployments on a Raspberry Pi 4 and an NVIDIA Jetson Nano, including the measured latency, memory, power, and thermal numbers, the wireless scan-to-report pipeline, and what still needs a human in the loop.

Raspberry Pi 4 (4GB RAM, CPU only) • NVIDIA Jetson Nano (4GB RAM, 128-core Maxwell CUDA GPU) • Colab T4 GPU comparison • 20 of 20 demo images correct • No thermal throttling observed on either board

## 1 Overview

The BRISC 2025 dataset provides four classes of brain MRI slice: glioma, meningioma, pituitary tumor, and no tumor, with paired segmentation masks for the three tumor classes. This project trains classification and segmentation baselines on that dataset, then answers a question the dataset paper does not: what does it actually cost, in latency, memory, power, and heat, to run these models on cheap edge hardware instead of a workstation GPU – and, since not all edge hardware is the same, how a CPU-only board (the Pi) compares against a small CUDA-capable board (the Jetson Nano).

The deployed system is a two stage pipeline. A classifier decides what kind of scan it is looking at. If the classifier is confident it is looking at a tumor, a segmenter runs next and marks where the tumor is. Every run produces a PDF report. The whole thing has been run and measured on real hardware – a Raspberry Pi 4 and a Jetson Nano – not estimated.

## 2 Models and Checkpoints

Two classifiers and two segmenters were trained (see `train_colab.ipynb`), so the deployment can pick a pair by size or by accuracy. The segmenters share a ResNet34 encoder; only the decoder differs.

| Checkpoint | Role | File size | Parameters | Reported accuracy |
|---|---|---|---|---|
| EfficientNetB0 | Classifier | 15.59 MB | ∼5.3M | 99.20% F1 |
| MobileNetV3 Large | Classifier | 16.25 MB | ∼5.5M | 94.47% F1 |
| DeepLabV3+ (ResNet34) | Segmenter | 85.77 MB | ∼24.4M | 76.3% mIoU |
| UNet (ResNet34) | Segmenter | 93.38 MB | ∼24.4M | 75.7% mIoU |

Table 1: Checkpoint size, parameter count, and reported accuracy by role.

The deployed app defaults to EfficientNetB0 and DeepLabV3+, the pair with the best reported accuracy on each task. That default is not the fastest pair on either edge board; see Sections 6 and 7 for why that trade is still worth keeping.

## 3 Inference Pipeline

Segmentation only runs when the classifier is actually confident it is looking at a tumor. This keeps the common case (no tumor, or a low confidence read) cheap, and avoids drawing a mask on an image the model was never sure about in the first place.

```
1. Classify   224x224 input, ImageNet normalization -> softmax over 4 classes
2. Gate       top class is a tumor type AND confidence >= 70% ?
                      |
          yes ------- + ------- no
           |                     |
3. Segment                    skip -> verdict is "no tumor" or "uncertain"
   256x256 input,
   binary mask
         |
4. Report     verdict, confidence chart, mask overlay,
              tumor area, run metadata -> PDF
```

The confidence threshold (default 70%) is adjustable. A predicted tumor class below threshold is reported as uncertain, not folded into either a confident tumor or a confident no-tumor call, and segmentation is skipped either way.

## 4 Edge Deployment Targets

| Board | RAM | OS | CPU | Accelerator |
|---|---|---|---|---|
| Pi 4 Model B | 4 GB | Raspberry Pi OS Full (Bookworm, 64-bit) | 4 threads | none (CPU only) |
| Jetson Nano | 4 GB | L4T / JetPack 4.6.6 (R32.7.6, Ubuntu 18.04) | 4x Cortex-A57 | Tegra X1, 128-core Maxwell CUDA |

Table 2: Target hardware and OS configuration.

64-bit Raspberry Pi OS is a hard requirement for the Pi: official PyTorch wheels only target aarch64 Linux, and 32-bit armhf has no practical modern PyTorch build. On the Jetson, the constraint runs the other way: the generic PyPI PyTorch wheel (CPU-only, the same one used for the Pi) is *not* built for Jetson's ARM+CUDA combination, so it either fails to install or silently installs without GPU support. The Jetson instead needs NVIDIA's own JetPack-matched wheel, plus torchvision built from source against it (no prebuilt torchvision wheel exists for this combination).

```
# Pi 4 (CPU-only wheel)
pip install torch torchvision --index-url https://download.pytorch.org/whl/cpu

# Jetson Nano, JetPack 4.6.x / L4T R32.7.x (Python 3.6)
wget <NVIDIA JetPack-matched torch 1.10.0 wheel>
```

```
pip install torch-1.10.0-cp36-cp36m-linux_aarch64.whl
git clone --branch v0.11.1 https://github.com/pytorch/vision torchvision
cd torchvision && python3 setup.py install
```

## 5 Resource Budget

Before anything was measured on either device, a budget was worked out on paper, then checked against what actually happened.

**Storage.** The four checkpoints together are about 213 MB. Add the OS and Python environment and the whole install lands around 3 to 4 GB on the Pi's 32 GB card, or a few GB more on the Jetson once the from-source torchvision build artifacts are included. Storage was never the binding constraint on either board.

**Memory.** The Pi's pre-deployment estimate (Table 3) put peak usage around 1.5 to 1.8 GB; the measured figure, with all four models loaded into one process at once, came in at 676 MB peak resident memory. The Jetson runs the identical all-four-models-loaded benchmark and measured **3023–3041 MB** peak RSS – far higher, because the Jetson build additionally holds CUDA context and GPU-resident weights/activations on top of the same Python/PyTorch overhead, out of only 3964 MB total system RAM. That is close enough to the ceiling that spawning even a small extra subprocess (e.g. `tegrastats`) while all four models are resident can fail with `OSError: Cannot allocate memory` on `fork()` – see Section 12.

| Component | Estimated |
|---|---|
| OS and services, headless | ∼500 MB |
| Python and the Streamlit server | ∼200 MB |
| PyTorch runtime import overhead | ∼250 MB |
| Model weights and activations | ∼500 MB |
| Chart and PDF rendering, transient | ∼100 MB |

Table 3: Pre-deployment memory budget estimate (Pi).

## 6 Measured Latency – Raspberry Pi 4

Measured with `benchmark.py`: 2 untimed warmup runs followed by 10 timed runs per stage, on a single 512x512 test slice, all four checkpoints loaded at once.

| Model | Mean | Median | P95 | Min | Max |
|---|---|---|---|---|---|
| Classifier: EfficientNetB0 | 282.0 | 280.1 | 308.7 | 274.8 | 308.7 |
| Classifier: MobileNetV3 | 189.3 | 187.2 | 207.5 | 186.2 | 207.5 |
| Segmenter: DeepLabV3+ | 1007.6 | 1004.5 | 1045.7 | 992.6 | 1045.7 |
| Segmenter: UNet | 926.5 | 924.3 | 938.4 | 918.6 | 938.4 |

Table 4: Per-model latency on the Pi 4, all figures in milliseconds.

| Combination | Mean | Median | P95 | Min | Max |
|---|---|---|---|---|---|
| EfficientNetB0 + DeepLabV3+ (default) | 1295.5 | 1292.7 | 1325.5 | 1288.7 | 1325.5 |
| EfficientNetB0 + UNet | 1203.9 | 1203.1 | 1223.0 | 1194.1 | 1223.0 |
| MobileNetV3 + DeepLabV3+ | 1196.3 | 1197.8 | 1200.5 | 1188.5 | 1200.5 |
| MobileNetV3 + UNet | 1114.6 | 1114.8 | 1125.0 | 1102.2 | 1125.0 |

Table 5: End-to-end pipeline latency on the Pi 4, all figures in milliseconds.

A full analysis, upload to result, lands around 1.1 to 1.3 seconds on the Pi.

## 7 Measured Latency – Jetson Nano

Measured with `benchmark_jetson.py` on the same methodology as the Pi (5 untimed warmup runs, 20 timed runs per stage, all four checkpoints loaded at once), on CUDA (Tegra X1 GPU, `torch.cuda.synchronize()` around every timed call since GPU kernel launches are asynchronous).

| Model | Mean | Median | P95 | Min | Max |
|---|---|---|---|---|---|
| Classifier: MobileNetV3 | 40.7 | 39.8 | 48.2 | 34.3 | 48.2 |
| Classifier: EfficientNetB0 | 51.4 | 50.5 | 59.3 | 46.2 | 59.3 |
| Segmenter: UNet | 100.7 | 100.3 | 105.1 | 99.2 | 105.1 |
| Segmenter: DeepLabV3+ | 111.7 | 111.6 | 113.7 | 111.1 | 113.7 |

Table 6: Per-model latency on the Jetson Nano (CUDA), all figures in milliseconds.

| Combination | Mean | Median | P95 | Min | Max |
|---|---|---|---|---|---|
| MobileNetV3 + UNet | 141.1 | 141.2 | 143.5 | 137.1 | 143.5 |
| EfficientNetB0 + UNet | 151.7 | 151.8 | 158.0 | 146.6 | 158.0 |
| MobileNetV3 + DeepLabV3+ | 151.8 | 152.0 | 154.8 | 149.0 | 154.8 |
| EfficientNetB0 + DeepLabV3+ (default) | 162.9 | 163.6 | 167.0 | 158.9 | 167.0 |

Table 7: End-to-end pipeline latency on the Jetson Nano, all figures in milliseconds.

A full analysis on the Jetson lands around 140–165 ms end to end – roughly **8–9x faster than the Pi** on the same default model pair (162.9 ms vs 1295.5 ms), for a device in the same rough price and power class, because the Nano's CUDA GPU can actually run these convolution-heavy models in parallel instead of walking them through 4 CPU threads. Peak RSS with all four models loaded was **3023–3041 MB** (see Section 5), against 676 MB on the Pi.

## 8 Jetson Power and Thermal Behavior

Captured with `tegrastats` running concurrently in a separate SSH session while `benchmark_jetson.py` ran the full latency sweep above.

| Metric | Value |
|---|---|
| Peak instantaneous power (`POM_5V_IN`) | 7309 mW (∼7.3 W), during a GR3D_FREQ 99% segmenter pass |
| Peak GPU power (`POM_5V_GPU`) | 4363 mW (∼4.4 W), same spike |
| Steady running-average power | ∼2.4–2.7 W |
| Idle baseline power | ∼0.9–1.5 W |
| Peak thermal (AO-therm) | 48.5°C – no throttling |
| CPU clock under load | up to 1479 MHz sustained |

Table 8: Jetson Nano power draw and thermal peaks over the full benchmark run.

The Jetson's thermal zones (AO, CPU, GPU, PLL, PMIC, and an iwlwifi radio zone) all stayed well clear of any throttle point through the run – the highest reading observed anywhere was 48.5°C, versus a typical throttle point in the 90s for this SoC. Power draw tracks GPU utilization tightly: every segmenter pass (`GR3D_FREQ` 99%) produces a sharp instantaneous spike toward 5–7 W, while idle and classifier-only periods sit under 2 W.

## 9 Pi versus Jetson versus T4 GPU

The same four combinations were run with the same methodology on a Colab T4 GPU (`colab_gpu_benchmark.ipyn` with `torch.cuda.synchronize()` around every timed call. The Jetson numbers use the clean (uncontended) run from Section 7.

| Model | Pi 4 (CPU) | Jetson Nano (CUDA) | T4 (CUDA) | Pi → Jetson speedup |
|---|---|---|---|---|
| Classifier: EfficientNetB0 | 282.0 ms | 51.4 ms | 8.76 ms | 5.5x |
| Classifier: MobileNetV3 | 189.3 ms | 40.7 ms | 6.10 ms | 4.7x |
| Segmenter: DeepLabV3+ | 1007.6 ms | 111.7 ms | 9.76 ms | 9.0x |
| Segmenter: UNet | 926.5 ms | 100.7 ms | 6.51 ms | 9.2x |

Table 9: Mean per-model latency across all three platforms.

The Jetson Nano sits where its hardware suggests it should: a genuine CUDA GPU (even a small, several-generations-old Maxwell part) closes most of the gap to a datacenter T4 on the compute-heavy segmenters, while still being ∼10–30x slower than the T4 itself. Segmentation again gains more from having any GPU at all (9.0–9.2x Pi-to-Jetson) than classification does (4.7–5.5x), consistent with the Pi-vs-T4 finding in the original report: the heavier ResNet34-encoder segmenters have more to parallelize than the lighter classifiers.

Read this three-way comparison as directional, not exact – as with the original Pi/T4 comparison, the three runs used different test images and run counts.

## 10 Correctness Check

Speed is only useful if the answers are right. All 20 demo images (5 per class) were run through the default pair on the T4 GPU; every prediction matched its known class, every tumor image produced a segmentation with a plausible tumor area, and every no-tumor image correctly skipped segmentation.

| Class | Predicted | Confidence range | Result |
|---|---|---|---|
| Glioma (5 images) | glioma | 92.1% to 100.0% | segmented |
| Meningioma (5 images) | meningioma | 99.6% to 100.0% | segmented |
| Pituitary (5 images) | pituitary | 99.9% to 100.0% | segmented |
| No tumor (5 images) | no_tumor | 100.0% | skipped |

Table 10: Prediction outcome on the 20-image demo set (T4 GPU, default pair).

## 11 Wireless Scan Pipeline

The deployment also covers getting a raw scan onto an edge board in the first place. This half runs on a separate local computer. Two send targets are supported – `send_to_pi.py` and `send_to_jetson.py` – sharing the same SSH/SCP plumbing (`ssh_utils.py`) so the pipeline works identically against either board.

```
.nii / .nii.gz scan
  | nii_to_slices.py, local computer
  v
PNG slices, anonymized sequential filenames
  | send_to_pi.py or send_to_jetson.py, SSH / SCP
  v
case_report.py / case_report_jetson.py runs on the target board
  | classifies every slice, ranks by confidence, segments only
  | the single best view, writes one consolidated PDF
  v
PDF report
  | SCP back to the local computer
  v
Downloaded report
```

`nii_web_app.py` wraps the upload, conversion, preview, and send steps (to either board) in a browser page so the local side needs no terminal. It requires SSH key based authentication, because a web app cannot answer an interactive password prompt; it fails fast with a clear message (including a host-key verification failure, the first time a given machine talks to a given board) instead of hanging if auth has not been set up yet.

## 12 Operational Notes

A short record of the real issues hit while wiring this up on actual hardware, kept here so the next setup does not repeat the same debugging.

### Raspberry Pi

- PyTorch 2.6 changed `torch.load`'s default. Loading a checkpoint that is not a bare tensor now fails unless `weights_only=False` is passed explicitly.
- Git LFS was not installed on the Pi at first; checkpoints pulled as 133 byte pointer stubs instead of real weights.

- Git Bash on Windows rewrites POSIX-looking paths in remote SSH command strings; `MSYS_NO_PATHCONV=1` on the subprocess environment stops it.

### Jetson Nano

- The generic PyPI PyTorch wheel is not built for Jetson's ARM+CUDA combination and must be replaced with NVIDIA's JetPack-matched wheel; torchvision has no matching prebuilt wheel at all and must be built from source against it.
- Older JetPack images ship Python 3.6 with an old pip/setuptools that has no prebuilt wheel for numpy, Pillow, or matplotlib on aarch64 – installing these via `apt` (`python3-numpy`, `python3-pil`, `python3-matplotlib`) and inheriting them into a `--system-site-packages` venv avoids a failed from-source compile.
- `torch.load`'s `weights_only` keyword does not exist on torch 1.10.0 (the JetPack 4.6.x wheel) – it was added in torch 1.13. Passing it raises `TypeError`; omitting it is behavior-preserving, since full unpickling was already torch 1.10's only mode.
- `subprocess.Popen(..., text=True)` is Python 3.7+ only; Jetson's stock Python 3.6 needs `universal_newlines=True` instead, which behaves identically.
- With all four models resident (peak RSS ∼3 GB out of 3964 MB total), spawning even a small subprocess like `tegrastats` can fail `fork()` with `OSError: [Errno 12] Cannot allocate memory` rather than the more familiar `FileNotFoundError` case (binary absent, e.g. off-Jetson) – both are now caught the same way, degrading to "power draw not captured" instead of crashing the benchmark.
- Running `tegrastats` concurrently in a second SSH session while the benchmark runs (rather than having the benchmark spawn it inline) avoids that memory contention and also avoids the two processes competing for CPU/GPU time, which was observed to roughly double tail latency (p95) versus an uncontended run.

## 13 Limitations

**Not a diagnostic device.** This is a research and demonstration system. Every report it produces says so. Findings should always be reviewed by a qualified radiologist before they inform any clinical decision.

**Domain mismatch is a real failure mode, not a hypothetical one.** The models were trained on BRISC: contrast-enhanced T1-weighted slices with the skull present. Running a scan from a different acquisition pipeline produced plausible-looking but untrustworthy classifications during testing. The pipeline does not know when it has been handed out-of-distribution data; it will answer confidently either way.

**Benchmark comparability.** The Pi, Jetson, and T4 timing runs were not a perfectly controlled A/B: different single test images, different run counts, and different hardware generations. Treat the speedup figures in Section 9 as representative, not as a certified benchmark result.